

Slayt 4 Ödevler

Ödev 1: Alfabedeki tüm karakterlerin string içinde kaç defa geçtiğini bulan uygulamayı yazın?

- Ödev 2: **Karakterin küçük harf olup olmadığını bulan uygulama.** Yukarıdaki uygulama Türkçe karakterler için doğru sonuç üretmemektedir. Fonksiyonu Türkçe karakterler için de çalışacak şekilde yeniden düzenleyin. lowercase değişkeninin string tipte bir değişken olduğunu göz önünde bulundurun.
- Ödev 3: Slayt 4 sonundaki ödevler

stringtools.py isimli bir Python dosyası içine aşağıda açıklamaları verilen fonksiyonları yazın ve çağırın?

```
def reverse(s):
    """
    >>> reverse('happy')
    'yppah'
    >>> reverse('Python')
    'nohtyP'
    >>> reverse("")
    ""
    >>> reverse("P")
    'p'
    """

def mirror(s):
    """
    >>> mirror("good")
    'gooddoog'
    >>> mirror("yes")
    'yessey'
    >>> mirror('Python')
    'PythonnohtyP'
    >>> mirror("")
    ""
    >>> mirror("a")
    'aa'
    """

def remove_letter(letter, strng):
    """
    >>> remove_letter('a', 'apple')
    'pple'
    >>> remove_letter('a', 'banana')
    'bnn'
    >>> remove_letter('z', 'banana')
    'banana'
    >>> remove_letter('i', 'Mississippi')
    'Msssspp'
    """
```

stringtools.py isimli bir Python dosyası içine aşağıda açıklamaları verilen fonksiyonları yazın ve çağırın?

```
def is_palindrome(s):
    """
    >>> is_palindrome('abba')
    True
    >>> is_palindrome('abab')
    False
    >>> is_palindrome('tenet')
    True
    >>> is_palindrome('banana')
    False
    >>> is_palindrome('straw warts')
    True
    """

def count(sub, s):
    """
    >>> count('is', 'Mississippi')
    2
    >>> count('an', 'banana')
    2
    >>> count('ana', 'banana')
    2
    >>> count('nana', 'banana')
    1
    >>> count('nanan', 'banana')
    0
    """
```

stringtools.py isimli bir Python dosyası içine aşağıda açıklamaları verilen fonksiyonları yazın ve çağırın?

```
def remove(sub, s):
    """
    >>> remove('an', 'banana')
    'bana'
    >>> remove('cyc', 'bicycle')
    'bile'
    >>> remove('iss', 'Mississippi')
    'Missippi'
    >>> remove('egg', 'bicycle')
    'bicycle'
    """

def remove_all(sub, s):
    """
    >>> remove_all('an', 'banana')
    'ba'
    >>> remove_all('cyc', 'bicycle')
    'bile'
    >>> remove_all('iss', 'Mississippi')
    'Mippi'
    >>> remove_all('eggs', 'bicycle')
    'bicycle'
    """
```

Slayt 6 Ödevler

Ödev 4: Aşağıdaki listede dolaşan ve her öğeyi ve indisini ekran görüntüsündeki gibi yazan programı yazın?

```
list=['spam!', 1, ['Brie', 'Roquefort', 'Pol le Veq'], [1, 2, 3]]
```

Ödev 5: Bir liste içindeki bir item'ın liste içindeki tüm index konumlarını bir döngü içinde bulup ekrana listeleyen Python programını yazın?

Ödev 6: Slayt 6 sonundaki ödevler

```
def add_lists(a, b):
```

```
    """
```

```
    >>> add_lists([1, 1], [1, 1])
```

```
    [2, 2]
```

```
    >>> add_lists([1, 2], [1, 4])
```

```
    [2, 6]
```

```
    >>> add_lists([1, 2, 1], [1, 4, 3])
```

```
    [2, 6, 4]
```

```
    """
```

```
def mult_lists(a, b):
```

```
    """
```

```
    >>> mult_lists([1, 1], [1, 1])
```

```
    2
```

```
    >>> mult_lists([1, 2], [1, 4])
```

```
    9
```

```
    >>> mult_lists([1, 2, 1], [1, 4, 3])
```

```
    12
```

```
    """
```

```
def add_row(matrix):
```

```
    """
```

```
    >>> m = [[0, 0], [0, 0]]
```

```
    >>> add_row(m)
```

```
    [[0, 0], [0, 0], [0, 0]]
```

```
    >>> n = [[3, 2, 5], [1, 4, 7]]
```

```
    >>> add_row(n)
```

```
    [[3, 2, 5], [1, 4, 7], [0, 0, 0]]
```

```
    >>> n
```

```
    [[3, 2, 5], [1, 4, 7]]
```

```
    """
```

```
def add_column(matrix):
```

```
    """
```

```
    >>> m = [[0, 0], [0, 0]]
```

```
    >>> add_column(m)
```

```
    [[0, 0, 0], [0, 0, 0]]
```

```
    >>> n = [[3, 2], [5, 1], [4, 7]]
```

```
    >>> add_column(n)
```

```
    [[3, 2, 0], [5, 1, 0], [4, 7, 0]]
```

```
    >>> n
```

```
    [[3, 2], [5, 1], [4, 7]]
```

```
    """
```

```
def add_matrices(m1, m2):
```

```
    """
```

```
    >>> a = [[1, 2], [3, 4]]
```

```
    >>> b = [[2, 2], [2, 2]]
```

```
    >>> add_matrices(a, b)
```

```
    [[3, 4], [5, 6]]
```

```
    >>> c = [[8, 2], [3, 4], [5, 7]]
```

```
    >>> d = [[3, 2], [9, 2], [10, 12]]
```

```
    >>> add_matrices(c, d)
```

```
    [[11, 4], [12, 6], [15, 19]]
```

```
    >>> c
```

```
    [[8, 2], [3, 4], [5, 7]]
```

```
    >>> d
```

```
    [[3, 2], [9, 2], [10, 12]]
```

```
    """
```

```
def scalar_mult(n, m):
```

```
    """
```

```
    >>> a = [[1, 2], [3, 4]]
```

```
    >>> scalar_mult(3, a)
```

```
    [[3, 6], [9, 12]]
```

```
    >>> b = [[3, 5, 7], [1, 1, 1], [0, 2, 0], [2, 2, 3]]
```

```
    >>> scalar_mult(10, b)
```

```
    [[30, 50, 70], [10, 10, 10], [0, 20, 0], [20, 20, 30]]
```

```
    >>> b
```

```
    [[3, 5, 7], [1, 1, 1], [0, 2, 0], [2, 2, 3]]
```

```
    """
```

#sıtrı çarpı sütun

```
def row_times_column(m1, row, m2, column):
```

```
    """
```

```
    >>> row_times_column([[1, 2], [3, 4]], 0, [[5, 6], [7, 8]], 0)
```

```
    19
```

```
    >>> row_times_column([[1, 2], [3, 4]], 0, [[5, 6], [7, 8]], 1)
```

```
    22
```

```
    >>> row_times_column([[1, 2], [3, 4]], 1, [[5, 6], [7, 8]], 0)
```

```
    43
```

```
    >>> row_times_column([[1, 2], [3, 4]], 1, [[5, 6], [7, 8]], 1)
```

```
    50
```

```
    """
```

```
def matrix_mult(m1, m2):
```

```
    """
```

```
    >>> matrix_mult([[1, 2], [3, 4]], [[5, 6], [7, 8]])
```

```
    [[19, 22], [43, 50]]
```

```
    >>> matrix_mult([[1, 2, 3], [4, 5, 6]], [[7, 8], [9, 1], [2, 3]])
```

```
    [[31, 19], [85, 55]]
```

```
    >>> matrix_mult([[7, 8], [9, 1], [2, 3]], [[1, 2, 3], [4, 5, 6]])
```

```
    [[39, 54, 69], [13, 23, 33], [14, 19, 24]]
```

```
    """
```

Slayt 7 Ödevler

Ödev 7: Dosyaya veri ekleme, silme vb.